

1. Purpose, Learning Outcomes, and Assessment Philosophy
 - 1.1 Alignment with the CO3117 Syllabus and Written Final Exam
 - 1.2 Release-Time Adaptation for HK261 (Official Calendar Schedule)
2. Non-Negotiable Rule: One Dataset, One Use Case
 - 2.1 Canonical Use Case and Approved Alternatives
3. Recommended Study Stack: Repositories, MOOC, and Textbooks
 - 3.1 Recommended Video MOOC: NPTEL Introduction to Machine Learning
 - 3.2 Reference Textbooks and Their Specific Roles
4. Required Learning Workflow for Every Major Model Family
 - 4.1 Implementation-Depth Classification
 - 4.2 Syllabus-to-Assignment Model Matrix
5. Repository Structure, Progress Dashboard, and Commit Standards
 - 5.1 PROGRESS.md: Course Dashboard
 - 5.2 Commit and Tagging Policy
6. Weekly Wiki/Blog Post: The Living Knowledge Dossier
7. Written-Exam Track and Handwritten Homework Addendum
 - 7.1 Weekly First-Attempt Drill
 - 7.2 Living Two-Page A4 Summary Sheet
 - 7.3 Final Mock Exam
 - 7.4 Handwritten Homework Sets (HW1 to HW5 Addendum)
8. Visual 15-Week Syllabus-Aligned Progress Plan
9. Common Experimental Protocol and Per-Model Record
10. Two Graded Submission Milestones and Ownership Defense
 - 10.1 Individual Ownership Checks
11. Evaluation Rubric (100-Point Allocation)
12. Academic Integrity & AI-Assisted Inquiry-Based Learning (IBL) Protocol
13. Final Synthesis Report Structure (6–8 Pages)

INDIVIDUAL LONGITUDINAL ASSIGNMENT

ONE DATASET, ONE USE CASE, MANY MODELS — A two-part theory → code → experiment → written-exam learning portfolio

```
<div class="banner-meta-item">
  <div class="banner-meta-label">Format</div>
  <div class="banner-meta-value">Individual • 1 Git Repo • 1 Dataset</div>
</div>
<div class="banner-meta-item">
  <div class="banner-meta-label">Workload</div>
  <div class="banner-meta-value">~45 Hours Total across 15 Weeks</div>
</div>
<div class="banner-meta-item">
  <div class="banner-meta-label">Part I Submission (40 pts)</div>
  <div class="banner-meta-value">Wed 14 Oct 2026 • 23:59 (Tag part1-final)</div>
</div>
<div class="banner-meta-item">
  <div class="banner-meta-label">Part II Submission (60 pts)</div>
  <div class="banner-meta-value">2 Days before Official Final Exam</div>
</div>
```

🔴 Core Design Principle: Fixed Problem, Evolving Models

Keep the dataset source, target variable, decision context, and held-out test population **strictly fixed for the entire semester**. Every week, change the model or feature representation—not the problem—so that differences in mathematical assumptions, optimization dynamics, inductive biases, and empirical inference can be compared rigorously and meaningfully.

```
<div class="info-card-header">
  <span>1. Prospective Ownership</span>
  <span class="badge badge-depth-a">Mandatory</span>
</div>
<div class="info-card-body">
  No retroactive back-dating or deadline code-dumps. Progression starts officially in <strong>Course Week 5</strong> with the two-state commit
</div>
```

1. Purpose, Learning Outcomes, and Assessment Philosophy

This assignment converts the traditional multi-model project into an individual, longitudinal learning system. Students repeatedly revisit the same prediction task with the model families in the CO3117 syllabus. The primary goal is not leaderboard optimization or metric chasing; the goal is to connect mathematical formulation, algorithmic steps, implementation choices, empirical behavior, and written explanation.

- **Explain** core ML concepts, assumptions, objectives, update rules, inference procedures, and model limitations in concise, rigorous written form.
- **Implement** selected core mechanisms from scratch in NumPy/Python (Depth A) and validate them numerically against trusted references.
- **Read** reference code critically (Depth B), mapping equations and algorithm steps to concrete code paths.
- **Compare** model families under a common split, preprocessing protocol, primary metric, and held-out test population.
- **Build** a cumulative, inspectable learning record through prospective Git commits, a wiki/blog knowledge base, weekly exercises, and two graded submission milestones.
- **Demonstrate** individual ownership: the student must be able to derive, locate, explain, test, modify, and defend any submitted component on demand.

1.1 Alignment with the CO3117 Syllabus and Written Final Exam

The CO3117 syllabus emphasizes understanding fundamental concepts, deriving mathematical foundations and training mechanisms of representative models, comparing strengths and limitations, applying appropriate algorithms, performing dimensionality reduction, and evaluating models in realistic settings.

The sample 90-minute final exam (code 504, 26 May 2026) uses short constructed-response questions rather than programming. It tests conceptual definitions, comparisons, conditional independence in Bayesian networks, PCA derivation, AdaBoost weight updates, generative versus discriminative modeling, and SVM margin reasoning. This assignment therefore establishes an integrated written-exam track alongside coding and controlled experiments.

⚠️ Pedagogical Principle

The assignment develops practical implementation ownership and written theoretical fluency simultaneously. **Code alone is not sufficient; prose alone is not sufficient.** You must be able to derive what you code, and explain without code what your models compute.

1.2 Release-Time Adaptation for HK261 (Official Calendar Schedule)

This specification takes effect in **Course Week 5**, corresponding to calendar week 39 (23 September 2026). The first class meeting occurred in calendar week 35; calendar week 36 (2 September) had no class due to the National Day holiday; calendar week 37 was the second teaching meeting (Decision Trees). Therefore, no student is expected or permitted to fabricate W01–W04 Git history, back-date commits, or claim historical pre-release attempts that did not exist.

```

<div class="info-card-header">
  <span>W01-W04 Pre-Release Protocol</span>
  <span class="badge badge-prerelease">Pre-Release</span>
</div>
<div class="info-card-body">
  <ul>
    <li><strong>Scope:</strong> Treated strictly as PRE-RELEASE content. Complete 01 concise <code>PRE_RELEASE_CATCHUP.md</code> plus 01 release
    <li><strong>Time Cap:</strong> Capped at 2-3 hours total. Date both artifacts honestly when created. Do not manufacture artificial history.
  </ul>
</div>

```

- **Course Week 8 is a protected MIDTERM week:** The midterm is scheduled for **Friday 16 October 2026**. Part I must be frozen and submitted on **Wednesday 14 October 2026 at 23:59**, exactly two calendar days earlier. Do not start a new major implementation or heavy experiment during the midterm window. After the exam, submit only a short reflection note within 72 hours; this belongs to Part II learning evidence and must not rewrite Part I history.
- **Release Checkpoint R0:** Due before Course Week 6 (Wednesday 07 October 2026 at 07:00 AM).
- **Two Graded Parts:** After R0, weekly checkpoints continue as continuous ownership evidence, but the assignment has only **TWO graded submission packages**:
 - **Part I:** Due **14 October 2026 at 23:59** (40 / 100 points).
 - **Part II:** Due **two calendar days before the official final examination** (60 / 100 points).
- **Index by Course Week:** All academic requirements in this document are indexed by **COURSE WEEK (W01-W15)** to match teaching meetings directly.

2. Non-Negotiable Rule: One Dataset, One Use Case

Mandatory Invariant Rule

The dataset source and version, prediction target, decision context, and held-out test population **remain fixed for the entire semester**. Different feature representations of the SAME raw data are encouraged; switching to a second dataset merely to accommodate a particular model is strictly prohibited.

- **Freeze Protocol:** Finalize the dataset source, target label, split policy, primary metric, and random seeds by the end of the Course Week 5 release checkpoint (R0), before Week 6 begins. Document this in `protocol.yaml`.
- **Pre-processing Hygiene:** Fit all scalers, encoders, PCA/LDA projections, and feature selectors on training data only; then transform validation and test sets.
- **Sealed Test Set:** Keep the final test partition sealed until cross-model comparison; all hyperparameter tuning occurs strictly on training/validation folds.
- **Sequential Views:** For sequential models (HMM/CRF), derive an ordered, temporally continuous view from the same underlying dataset and document the derivation.
- **Model Suitability Study:** If a syllabus model is genuinely mismatched with your fixed prediction task, submit a rigorous Model Suitability Study explaining the theoretical mismatch rather than forcing a scientifically meaningless run.

2.1 Canonical Use Case and Approved Alternatives

- **Canonical Dataset:** UCI Human Activity Recognition Using Smartphones (UCI HAR). The stable task is: *predict a user's current physical activity from smartphone inertial sensor measurements*. For HMM/CRF, temporal continuity is exploited to smooth or decode activity sequences over time.
- **Structural Exemplar:** The prior-cohort report *"Detect AI Generated Text"* serves as a **structural and architectural exemplar** for assembling a theory → code → experiment → reflection dossier. It is not an answer key. Students must not copy its text, code, tables, or results. If text detection is approved as a custom use case, a single dataset must be frozen and any sequence transformation justified explicitly.

3. Recommended Study Stack: Repositories, MOOC, and Textbooks

```

<div class="info-card-header">
  <span>ML-From-Scratch</span>
  <span class="badge badge-depth-a">Classical ML</span>
</div>
<div class="info-card-body">
  <strong>Primary code microscope</strong> for Decision Tree, Perceptron/MLP, Naive Bayes, GA, SVM, PCA/LDA, and Ensembles.<br>
  <em>Rule: Read only after committing your own first attempt. Trace equations → code lines.</em>
</div>

```

```

<div class="info-card-header">
  <span>ML-From-Scratch</span>
  <span class="badge badge-depth-a">Classical ML</span>
</div>
<div class="info-card-body">
  <strong>Primary code microscope</strong> for Decision Tree, Perceptron/MLP, Naive Bayes, GA, SVM, PCA/LDA, and Ensembles.<br>
  <em>Rule: Read only after committing your own first attempt. Trace equations → code lines.</em>
</div>

```

3.1 Recommended Video MOOC: NPTEL Introduction to Machine Learning

- **Primary Video Companion:** NPTEL/SWAYAM, *Introduction to Machine Learning*, Prof. Balaraman Ravindran, IIT Madras ([Course Link](#)).
- **Syllabus Match:** Matches CO3117 closely: bias-variance, logistic regression, LDA, perceptron, SVM, neural networks, backpropagation, MLE/MAP, decision trees, cross-validation, bagging/boosting, Naive Bayes, Bayesian networks, and HMMs.
- **Active Viewing Routine (3-2-1 Protocol):**
 1. *Before watching:* Write 3 questions or predictions.
 2. *While watching:* Pause at least twice to derive the next mathematical step before the lecturer explains it.
 3. *After watching:* Close video/notes and write 1 compact concept map from memory.

3.2 Reference Textbooks and Their Specific Roles

Course Text / Reference	Recommended Pedagogical Role
Müller & Guido (2017), <i>Introduction to Machine Learning with Python</i>	Practical classical ML, preprocessing, model behavior, evaluation, and scikit-learn benchmarking.
Chollet (2021), <i>Deep Learning with Python</i> , 2nd ed.	Neural network intuition, gradient descent dynamics, backpropagation mechanics.
Murphy (2022), <i>Probabilistic Machine Learning: An Introduction</i>	Primary theory reference for probability, Bayesian learning, graphical models, and inference.
Tom Mitchell (1997), <i>Machine Learning</i>	Foundational conceptual treatment: concept learning, decision trees, hypothesis spaces.
Stephen Marsland (2009), <i>Machine Learning: An Algorithmic Perspective</i>	Algorithmic implementation perspective; converting mathematics into NumPy code.
Cao Hoàng Trụ (2008), <i>Trí tuệ Nhân tạo = Thông minh + Giải thuật</i>	Vietnamese-language conceptual depth, AI context, and heuristic search foundations.

4. Required Learning Workflow for Every Major Model Family

<div class="step-number">1</div><div class="step-title">Setup</div><div class="step-desc">Install dependencies, set up environment, and download dataset.</div></div>

<div class="step-number">2</div><div class="step-title">Train Baseline</div><div class="step-desc">Implement a baseline model using standard libraries.</div></div>

<div class="step-number">3</div><div class="step-title">Analyze Results</div><div class="step-desc">Evaluate model performance and compare against baseline.</div></div>

<div class="step-number">4</div><div class="step-title">Experiment Variants</div><div class="step-desc">Test different hyperparameters and model architectures.</div></div>

<div class="step-number">5</div><div class="step-title">Write Report</div><div class="step-desc">Document findings, conclusions, and future work.</div></div>

<div class="step-number">6</div><div class="step-title">Explain</div><div class="step-desc">Update weekly post, MODEL_LOG, exam sheet. Commit and push weekly tag.</div>

4.1 Implementation-Depth Classification

Depth Level	Meaning	Minimum Verifiable Evidence
Depth A — BUILD	Author the core algorithm yourself in NumPy/Python without high-level estimator libraries.	Mathematical derivation; clean NumPy code; unit tests against synthetic cases; pre-reference and post-reference commits.
Depth B — DISSECT & MODIFY	Study an approved reference implementation, explain its architecture, and execute a non-trivial adaptation.	Exact code attribution; line-by-line mapping; meaningful diff; controlled experiment demonstrating effect of modification.
Depth C — APPLY & BENCHMARK	Use an approved reference library to evaluate applicability, parameter sensitivity, and limits.	Justified hyperparameters; controlled benchmark; confusion matrix; comprehensive error/limitation analysis.

4.2 Syllabus-to-Assignment Model Matrix

Syllabus Topic	Model Family	Target Depth	Required Core Artifact
Ch. 1	Foundations & Workflow	Analysis	Learning curves, bias-variance diagnosis, leakage check, data protocol.
Ch. 2	Decision Tree	Depth B (Catch-up)	Implement impurity/split routine; inspect tree stopping; evaluate pruning.
Ch. 3	Perceptron & MLP	Depth A	Explicit forward/backward pass in NumPy; numerical gradient check; capacity test.
Ch. 4	Naive Bayes	Depth A	Own Naive Bayes classifier; Laplace/Gaussian smoothing; assumption analysis.
Ch. 5	Genetic Algorithm	Depth B	Student-authored representation, fitness, selection, crossover, mutation for search.
Ch. 6	Bayesian Networks & HMM	HMM: Depth A BN: Depth C	Own Viterbi or Forward core; numpy-ml validation; d-separation reasoning.
Ch. 7	Support Vector Machines	Depth B + Depth C	Dissect hinge-loss/margin; linear vs RBF kernel; C-parameter sensitivity study.
Ch. 8	PCA & LDA	PCA: Depth A LDA: Depth C	Own PCA projection via SVD/eigensystem; reconstruction check; PCA vs LDA study.
Ch. 9	Ensembles & AdaBoost	Depth B + Depth C	Bagging vs boosting comparison; AdaBoost sample reweighting & weighted vote trace.
Ch. 10	Discriminative & Synthesis	Logistic: Depth A CRF: Depth C	Own multiclass logistic/softmax; MaxEnt connection; CRF vs HMM suitability study.

5. Repository Structure, Progress Dashboard, and Commit Standards

All work is maintained in one continuous GitHub repository assigned to the student (HCMUT-C03117-261/mL-<student-id>).

```
co3117-ml-individual/
├─ README.md           Overview and project navigation
├─ PROGRESS.md         Weekly progress dashboard (verified by tools/check.py)
├─ MODEL_LOG.md        Cumulative catalog of evaluated models and metrics
├─ AI_USE.md           Mandatory log of AI interactions and inquiry trails
├─ REFERENCES.md       Full bibliographic and code repository citations
├─ SUBMISSION_PART1.md Part I executive summary and package manifest
├─ SUBMISSION_PART2.md Part II final report summary and cross-model synthesis
├─ protocol.yaml       Frozen dataset specifications and splitting policies
├─ homework/          Handwritten homework sheets (HW1-HW5) and guides
│   ├── hw1-problems.pdf ... hw5-problems.pdf
│   └─ hw1-guide.md ... hw5-guide.md
├─ docs/
│   ├── index.md       Knowledge base index
│   ├── pre-release/
│   │   └─ PRE_RELEASE_CATCHUP.md W01-W04 consolidated catch-up post
│   └─ weekly/
│       ├── w05-perceptron-ann.md W05 theory and experiment post
│       ├── w06-naive-bayes.md    W06 theory and experiment post
│       └─ ... w15-synthesis.md
├─ exercises/
│   ├── release-baseline-w01-w04.pdf Release diagnostic scan (Foundations & Decision Tree)
│   ├── w05-first-attempt.pdf       Weekly handwritten drill scans
│   └─ w05-corrections.md          Weekly markdown corrections
├─ exam/
│   ├── a4-notes-part1-midterm.pdf  Part I living two-page A4 exam sheet
│   ├── midterm-reflection.md       Post-midterm error analysis and reflection
│   ├── a4-notes-part2-final.pdf    Part II final two-page A4 exam sheet
│   └─ mock-final-first-attempt.pdf 90-minute mock final exam paper
├─ src/                          NumPy from-scratch and adapter implementations
├─ experiments/                  Controlled experiment scripts
├─ results/                      Generated figures, logs, and metrics.csv
├─ tools/
│   ├── check.py                 Automated repository self-check script
│   └─ course.yaml               Course calendar, deadlines, and checkpoint configuration
```

5.1 PROGRESS.md: Course Dashboard

PROGRESS.md contains one row per Course Week. tools/check.py validates every row up to the current active week.

Period	Topic	Post	Drill	First Evidence	Revision Commit	Tag	Status
W01-W04	PRE-RELEASE Catch-up (Foundations, Tree)	catch-up	baseline	(current date)	(catch-up)	release-baseline	PRE-RELEASE
W05	Perceptron & ANN + Repo Onboarding	post	drill	hash1	hash2	w05	ACTIVE
W06	Bayesian Learning / Naive Bayes	post	drill	hash1	hash2	w06	Scheduled
W07	Genetic Algorithms & Bayesian Networks	post	drill	hash1	hash2	w07 , part1-final	Scheduled
W08	MIDTERM (16 Oct 2026)	reflection	rehearsal	n/a	n/a	w08-midterm	MIDTERM
W09-W15	Part II Models & Final Synthesis	(weekly posts)	(weekly drills)	hashes	hashes	w09 .. w15 , part2-final	Scheduled

5.2 Commit and Tagging Policy

- Two-State Rule:** Each active week preserves: (i) first attempt before reference code / AI assistance, and (ii) corrected/extended state after testing and benchmarking.
- Commit Message Convention:** Tag commits with standard academic tags: [W05][baseline] release setup , [W05][code] own perceptron forward pass , [W06][theory] Naive Bayes conditional independence derivation .
- Git Tags:** Create tag release-baseline at R0, then weekly tags w05 , w06 , ..., w15 . Graded milestones are tagged part1-final (14 Oct 23:59) and part2-final . Do not fabricate historical w01-w04 tags.

Course Week & Date	Curriculum Topic	Repository & Empirical Evidence	Written Drill & Assessment Milestones
W01 Pre-Release 26 Aug (Cal 35)	Foundations of ML Metrics, Bias-Variance, Overfitting	RETROSPECTIVE ONLY. Consolidated baseline pipeline, data leakage audit checklist, train/val learning curve diagnosis in catch-up post.	Covered in release-day W01–W04 baseline diagnostic: definitions, generalization bounds, curve diagnosis.
W02–W04 Pre-Release 09–16 Sep (Cal 37–38)	Decision Trees Entropy, Information Gain, Pruning	RETROSPECTIVE ONLY. Implement 1 impurity/split routine in NumPy; inspect reference tree; stopping/pruning comparison.	Covered in release-day baseline: Information gain calculation, continuous threshold splits, pruning logic.
W05 — RELEASE Active Start 23–30 Sep (Cal 39–40)	Perceptron, Delta Rule & Neural Networks (ANN)	PROSPECTIVE EVIDENCE STARTS. Initialize repo skeleton, <code>protocol.yaml</code> , <code>PROGRESS.md</code> , <code>AI_USE.md</code> . Implement own Perceptron and small MLP with explicit forward/backward pass. Tag: <code>release-baseline</code> , <code>w05</code> .	Written Drills: (A) Perceptron geometric update. (B) Linear separability & XOR failure. (C) Chain rule & backpropagation dynamics. HW1, HW2, HW3 due 07 Oct 07:00 AM.
W06 Part I 30 Sep–07 Oct (Cal 40)	Bayesian Learning & Naive Bayes	Author own Naive Bayes classifier in NumPy. Laplace smoothing and Gaussian feature modeling. Benchmark against scikit-learn. Tag: <code>w06</code> .	Written Drill: (A) Bayes theorem derivation. (B) Conditional independence assumption. (C) Zero-frequency smoothing. (D) Generative vs discriminative reasoning.
W07 Part I Closeout 07–14 Oct (Cal 41)	Genetic Algorithms & Bayesian Networks (TAN)	GA chromosome representation, fitness function, selection/crossover/mutation. Small BN/TAN factorization and d-separation checks. Consolidate Part I portfolio. Tag: <code>w07</code> .	Written Drill: BN joint factorization, d-separation under evidence, NB vs TAN. HW4, HW5 due 14 Oct 23:59. PART I FINAL DUE: 14 Oct 23:59 (Tag: <code>part1-final</code>).
W08 — MIDTERM Protected Week 16 Oct 2026	MIDTERM EXAMINATION Protected Written Exam Week	NO NEW MODEL IMPLEMENTATIONS. Part I is sealed. Complete timed exam rehearsal before exam; post-midterm reflection note within 72h. Tag: <code>w08-midterm</code> .	Formal written midterm exam covering syllabus Chapters 1 to 6. Focus on concise proofs, derivations, and model comparisons.
W09 Part II 21–28 Oct (Cal 43)	Hidden Markov Models (HMM) Sequence Modeling	Implement own Viterbi or Forward algorithm in NumPy. Validate on synthetic sequence; dissect <code>numpy-ml</code> ; compare independent vs sequence-aware predictions. Tag: <code>w09</code> .	Written Drill: (A) HMM components & joint factorization. (B) Forward recursion steps. (C) Viterbi decoding dynamic programming.
W10–W11 Part II 28 Oct–11 Nov (Cal 44–45)	Support Vector Machines (SVM) Linear & Kernel Methods	Dissect maximum-margin formulation and hinge loss. Linear vs RBF kernel comparison. Parameter sensitivity (C, γ) on fixed dataset. Tags: <code>w10</code> , <code>w11</code> .	Written Drill: (A) Support vectors and margin width. (B) Dual representation & Kernel trick. (C) SVM vs Logistic Regression comparison.
W12–W13 Part II 11–25 Nov (Cal 46–47)	Dimensionality Reduction PCA & LDA	Implement own PCA via SVD/eigensystem in NumPy. Reconstruction error analysis. Compare unsupervised PCA against supervised LDA class-separation. Tags: <code>w12</code> , <code>w13</code> .	Written Drill: (A) Curse of dimensionality. (B) Variance maximization vs decorrelation. (C) Fisher criterion and max LDA dimensions.
W14 Part II 25 Nov–02 Dec (Cal 48)	Ensemble Learning Bagging, Boosting, AdaBoost	Bagging vs Boosting benchmark on fixed dataset. Step-by-step trace of AdaBoost sample reweighting and weighted voting. Tag: <code>w14</code> .	Written Drill: (A) Variance reduction vs bias reduction. (B) AdaBoost weight update derivation. (C) Margin explanation in boosting.
W15 — SYNTHESIS Final Package 02–09 Dec (Cal 49)	Generative vs. Discriminative, CRF & Final Synthesis	Author multiclass Logistic Regression / Softmax in NumPy. Model suitability study for CRF. Assemble Master Comparison Table and final report. Complete 90-min mock final. Tag: <code>w15</code> .	Written Drill: Comprehensive comparative review. PART II FINAL DUE: 2 calendar days before official final exam date (Tag: <code>part2-final</code>).

9. Common Experimental Protocol and Per-Model Record

To ensure scientific rigor across all 15 weeks, every experiment must adhere to the standardized configuration defined in `protocol.yaml`:

- Primary Metric:** Macro-F1 score for multiclass classification. Always report Accuracy and Confusion Matrix as secondary evidence.
- Split Policy:** Subject-aware / group-aware splitting to prevent data leakage across train, validation, and test partitions.
- Controlled Tuning:** Hyperparameter optimization is performed strictly within training/validation sets. The test partition remains sealed.
- Failure Analysis:** For every major model family, inspect and document at least 5 representative classification errors or 1 systematic failure pattern.

10. Two Graded Submission Milestones and Ownership Defense

<div class="info-card-header">
 Part I: Pre-Midterm Portfolio (40 pts)
 Due 14 Oct 23:59
</div>
<div class="info-card-body">
 Includes W01–W04 catch-up, W05 Perceptron/ANN, W06 Naive Bayes, W07 GA & BN/TAN, weekly posts, written drills, HW1–HW5, <code>SUBMISSION_PART</code>
</div>

10.1 Individual Ownership Checks

The instructor may select any submitted artifact at random and request an impromptu individual verification:

- Derive the core equation or algorithm on a whiteboard without reference material.
- Locate the exact function and line in your code and explain its mapping to theory.
- Live-modify a parameter or function and predict its empirical effect prior to running.
- Answer an exam-style constructed-response theoretical question orally.
- Explain the precise conceptual progression between your first-attempt and final commit.

11. Evaluation Rubric (100-Point Allocation)

Evaluation Criterion	Pts (P1 + P2)	Full-Credit Standards
1. Weekly Progress & Git Ownership	12 (5 + 7)	Honest release baseline + regular weekly checkpoints; clean two-state commit history; standard academic tags; no back-dating or last-minute code dumps.
2. Weekly Wiki/Blog Knowledge Dossier	13 (6 + 7)	Rigorous concept capsules, derivations, code-to-theory traces, focused experiments, and reflection entries.
3. Written-Exam Portfolio & Homework	15 (8 + 7)	Release baseline diagnostic, weekly handwritten drills, HW1–HW5 completed, living 2-page A4 sheets, and 90-min final mock paper.
4. Required From-Scratch Implementations	18 (8 + 10)	Depth-A algorithms authored in NumPy without high-level estimator libraries, fully unit-tested and verified.
5. Reference-Repository Dissection & Adaptation	10 (4 + 6)	Line-by-line mapping of ML-From-Scratch/numpy-ml; non-trivial student modifications demonstrating clear understanding.
6. Experimental Design & Controlled Benchmarking	12 (4 + 8)	Fixed dataset invariants, leak-free preprocessing, fair benchmarking against scikit-learn, sealed test sets.
7. Evaluation, Error Analysis & Limitations	10 (3 + 7)	Meaningful metric interpretation, confusion matrix diagnostics, documented failure modes, model trade-offs.
8. Reproducibility & Code Quality	5 (1 + 4)	Reproducible seeds, modular code structure, environment definition, automated test compliance.
9. Final Synthesis Report & Academic Discipline	5 (1 + 4)	Concise synthesis (6–8 pages), cross-model comparison, transparent AI disclosure, proper citations.
TOTAL	100 (40 + 60)	Converted directly to course assignment weight in CO3117 syllabus.

12. Academic Integrity & AI-Assisted Inquiry-Based Learning (IBL) Protocol

AI assistants and external repositories may support learning only when they stimulate independent questioning, hypothesis formulation, and rigorous derivation. Productive struggle must always precede tool consultation.

- | | |
|----------------|---|
| 0. FRAME | Write learning question and initial hypothesis before opening AI. |
| ↓ | |
| 1. ATTEMPT | Solve/derive/code independently. Commit raw attempt to Git history. |
| ↓ | |
| 2. INQUIRE | Ask AI for ONE Socratic hint, diagnostic question, or counterexample at a time. |
| ↓ | |
| 3. VERIFY | Cross-check AI responses against lecture notes, Murphy, or Ravindran M00C. |
| ↓ | |
| 4. RECONSTRUCT | Close AI window. Re-derive or re-code from memory in your own structure. |
| ↓ | |
| 5. TRANSFER | Solve a parameter variation or edge case not present in the original prompt. |
| ↓ | |
| 6. TEACH-BACK | Explain the concept concisely as if teaching a peer (exam capsule). |
| ↓ | |
| 7. RETRIEVAL | Reproduce key derivation after 24–72 hours closed-book. |

- **Prohibited Uses:** Generating first-attempt drills, pasting AI code into Depth-A components, generating synthetic weekly posts, or disguising external solutions.
- **Disclosure:** Every AI interaction is recorded in `AI_USE.md` (Date, Learning Question, Pre-AI Commit, AI Tool, Prompt Purpose, Hint Summary, Verification Source, Closed-Book Status).

13. Final Synthesis Report Structure (6–8 Pages)

1. **Problem & Dataset (0.5 page):** Fixed use case, target variable, split policy, leakage controls.
2. **Representations & Protocol (0.5 page):** Static vs sequential representations, tuning protocols.
3. **Model-Family Map (1–1.5 pages):** Compact mathematical formulation and implementation depths.
4. **Empirical Results (1–1.5 pages):** Master Comparison Table and most diagnostic curves.
5. **Error & Limitation Analysis (1 page):** Systematic confusion patterns and inductive bias effects.
6. **Cross-Model Synthesis (1 page):** Generative vs discriminative, parametric vs non-parametric, bias vs variance.
7. **Ownership & Learning Reflection (0.5 page):** Major conceptual shifts and corrected misconceptions.
8. **Reproducibility & References (0.5 page):** Execution commands, environment, full citations.